# A General Framework for Encoding Financial Decision Structures into QUBO Models

**Molly Maskrey**
Lead Data Scientist, Forte Analytics
mmaskrey@forteinformation.com

November 2025

**Abstract**

This whitepaper presents a universal framework for converting real-world financial inputs into Quadratic Unconstrained Binary Optimization (QUBO) models suitable for classical annealing and quantum-hybrid solvers. While two illustrative examples are drawn from pharmaceutical rebate contracting, the framework is intentionally *industry-agnostic*.

Six end-to-end encoding examples are provided across retail fuel pricing, airline seat revenue management, manufacturing production planning, portfolio optimization, and pharmacy benefit contracting. Each example includes raw business inputs (e.g., Excel/CSV fields), binary encodings, objective construction, constraint penalties, and a leadership-friendly interpretation.

All methods are compatible with the DAYSTROM node (CPU+GPU) and the D-Wave Ocean SDK, using modern samplers such as `dwave.samplers.SimulatedAnnealingSampler`.

## Contents

**1 Introduction** **3**

**2 Universal Encoding Pipeline** **3**

**3 Binary Encoding of Financial Variables** **4**

**4 Building Objectives and Penalties** **4**

**5 Six Worked Examples** **5**
5.1 Example 1: Gas Station Pricing Optimization . . . . . 5
5.2 Example 2: Airline Seat Revenue Management . . . . . 5
5.3 Example 3: Manufacturing Production Allocation . . . . . 6
5.4 Example 4: Portfolio Optimization (10 Assets) . . . . . 7
5.5 Example 5 (PBM): Rebate Step Function . . . . . 8
5.6 Example 6 (PBM): GTN Floor + Copay Budget . . . . . 8

**6 Implementation Using Ocean SDK** **9**
6.1 Python Example . . . . . 9
6.2 Advanced Constraint Handling . . . . . 9

**7 Deployment Considerations** **11**
7.1 DAYSTROM Node Configuration . . . . . 11
7.2 Performance Optimization . . . . . 11

**8 Conclusion** **11**

# 1 Introduction

Many financial and operational decisions—pricing, budgeting, volume planning, contracting, or risk allocation—ultimately come down to choosing values that balance profitability, efficiency, and constraints. Decision makers interact with:

- prices, percentages, margins, volumes,
- budgets, risk thresholds,
- competitive dynamics and regulatory limits.

By contrast, a QUBO solver sees only:

$$E(\mathbf{q}) = \sum_i Q_{ii} q_i + \sum_{i<j} Q_{ij} q_i q_j, \qquad q_i \in \{0, 1\}.$$

The goal of this paper is to bridge those two worlds in a way that is:

- **technically rigorous** for reviewers familiar with optimization,
- **approachable** for leadership who may conflate "QPU" and "QUBO",
- **broadly applicable**, not limited to a single industry,
- **immediately implementable** using standard Python + Ocean workflows.

The methods herein have been deployed on the DAYSTROM hybrid node at Forte Analytics for pricing, contracting, and optimization research.

# 2 Universal Encoding Pipeline

The standard workflow for converting spreadsheet-style numbers into QUBO-ready variables follows these steps:

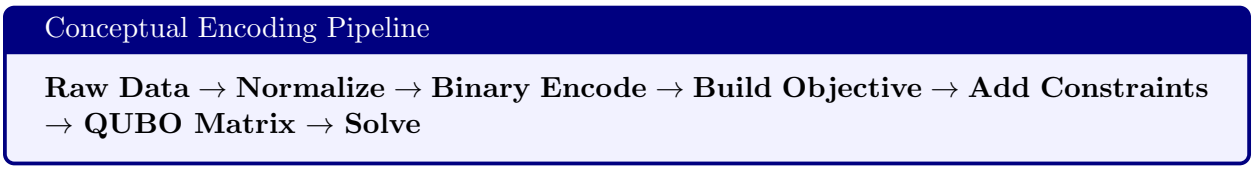



Figure 1: Conceptual encoding pipeline for financial QUBO models.

The pipeline consists of:

1. **Ingest**: Read raw inputs from Excel, CSV, email, internal databases.
2. **Normalize**: Convert units (USD, %, basis points), align periods (monthly, annual), define bounds.
3. **Binary Encoding**: Represent each decision variable as
$$\hat{x} = \sum_i w_i q_i.$$
4. **Objective Construction**: Translate revenue, cost, margin, or risk equations.

5. **Constraint Penalties**: Convert inequalities into penalty squares.

6. **QUBO Assembly**: Build matrix $Q$ using Ocean's `BinaryQuadraticModel`.

7. **Solve**: Run classical annealing locally or submit to a hybrid/QPU solver.

**Leadership Translation:**
We take the numbers you already work with—prices, costs, volumes, rebates—and convert them into a series of "switches" (bits). The solver flips these switches to find combinations that respect all your rules and deliver the best financial outcome.

## 3 Binary Encoding of Financial Variables

Let $x$ be a financial variable such as price, volume, or allocation percentage. We choose weights $w_i$ and binary variables $q_i$ such that:

$$\hat{x} = \sum_{i=0}^{n-1} w_i q_i.$$

The most common choice is **binary-weighted encoding**:

$$w_i = 2^i,$$

providing 0 to $2^n - 1$ coverage.

Scaling is often used to maintain numerical stability:

$$x' = s \cdot x.$$

## 4 Building Objectives and Penalties

Most financial optimization problems take the form:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \underbrace{\text{economic cost/benefit}}_{\text{objective}} + \underbrace{\text{constraint penalties}}_{\text{rules we must follow}}.$$

Equality constraints use:

$$\lambda(g(\mathbf{x}) - T)^2,$$

while inequality constraints use:

$$\lambda \max(0, g(\mathbf{x}) - T)^2.$$

This converts rules like budgets, floors, caps, and share requirements into "financial guardrails" inside the QUBO.

# 5 Six Worked Examples

We now present six fully worked encoding examples. The first four are industry-general, and the final two illustrate PBM contracting.

## 5.1 Example 1: Gas Station Pricing Optimization

**Raw Inputs (from Excel)**

| Field | Example Value |
|---|---|
| Wholesale cost (per gallon) | \$2.38 |
| Competitor average price | \$3.19 |
| Max allowed retail price | \$3.49 |
| Min margin requirement | \$0.25 |
| Expected daily volume | 4,500 gallons |

**Binary Encoding**

Let price increments be 1 cent. Represent retail price $P$ as:

$$P = P_{\min} + 0.01 \cdot (q_0 + 2q_1 + 4q_2 + 8q_3 + 16q_4).$$

**Objective Function**

Daily profit:

$$\Pi(P) = (P - C_{\text{wholesale}}) \cdot V.$$

**Constraints**

$$P \leq 3.49, \qquad P - C_{\text{wholesale}} \geq 0.25.$$

Penalties:

$$\lambda_1 \max(0, P - 3.49)^2, \qquad \lambda_2 \max(0, 0.25 - (P - C_{\text{wholesale}}))^2.$$

**Leadership Translation:** The solver chooses a pump price that balances profit and margin rules. If the price is too low (margin too small) or too high (violating policy), the configuration becomes unattractive and is rejected.

## 5.2 Example 2: Airline Seat Revenue Management

**Inputs (CSV)**

| Field | Value |
|---|---|
| Base fare range | \$120–\$260 |
| Load factor target | 87% |
| Competitor fare | \$199 |
| Max discount allowed | 15% |
| Seasonality factor | 0.92 |

**Encoding Fare**

Let fare $F$ use 5 bits with \$2 increments.

**Objective**

Expected revenue:

$$R(F) = F \cdot \mathbb{E}[\text{seats sold}(F)].$$

We encode a simple linear load-factor model or a lookup table based on historical data.

**Constraints**

$$F \geq 0.85 \cdot F_{\text{base,max}}, \qquad \text{and} \qquad F \leq F_{\text{competitor}} + \Delta.$$

**Leadership Translation:** We're letting the solver choose a fare that balances competitiveness with revenue. Constraints prevent giving away seats too cheaply.

### 5.3 Example 3: Manufacturing Production Allocation

**Inputs**

| Field | Value |
|---|---|
| Prod cost (normal) | \$14.00/unit |
| Overtime cost | \$21.00/unit |
| Storage cost | \$0.30/unit/day |
| Max warehouse capacity | 60,000 units |
| Demand forecast | 50,000 units |

**Encoding Production**

Let $X$ = units produced:

$$X = 5000(q_0 + 2q_1 + 4q_2 + 8q_3).$$

**Objective**

$$\text{Cost}(X) = C_{\text{prod}} X + C_{\text{store}} \max(0, X - D).$$

**Constraint**

$$X \leq 60{,}000.$$

**Leadership Translation:** This enforces "produce enough but don't overproduce." The solver picks the cheapest feasible output level.

### 5.4 Example 4: Portfolio Optimization (10 Assets)

**Inputs**

| Field | Description |
|---|---|
| $\mu_i$ | Expected return |
| $\sigma_{ij}$ | Covariance matrix |
| Budget | 100% |
| Min allocation per asset | 0–20% |

**Encoding Positions**

Let each allocation be in 2% increments:

$$x_i = 0.02(q_{i1} + 2q_{i2} + 4q_{i3}).$$

**Objective**

$$f(\mathbf{x}) = -\sum_i \mu_i x_i + \gamma \sum_{i,j} \sigma_{ij} x_i x_j.$$

**Constraint**

Budget:

$$\sum_i x_i = 1.$$

Penalty:

$$\lambda(\sum_i x_i - 1)^2.$$

**Leadership Translation:** The solver chooses an allocation balancing return (reward) and risk (penalty), while respecting the total 100% budget.

### 5.5 Example 5 (PBM): Rebate Step Function

**Inputs (Simple Rebate Case)**

| Field | Value |
|---|---|
| Base rebate range | 0–7% |
| Volume tier | 80,000 Rx threshold |
| Tier increment | +3% |
| WAC | $300 |
| Volume | 100,000 Rx |

**Encoding Rebate**

$$r = q_0 + 2q_1 + 4q_2.$$

Tier flag $I$:

$$I = 1 \Rightarrow V \geq 80{,}000.$$

**Objective: Rebate Cost**

$$C(r, I) = V \cdot \text{WAC} \cdot \frac{r + 3I}{100}.$$

**Constraint**

$$I \cdot \max(0, 80{,}000 - V)^2.$$

**Leadership Translation:** The solver chooses rebate levels that balance cost and access. Tiered rebates activate only when volume is high enough.

### 5.6 Example 6 (PBM): GTN Floor + Copay Budget

**Inputs**

| Field | Value |
|---|---|
| List price $P$ | variable |
| Rebate $R$ | variable |
| Copay support $C$ | variable |
| Required GTN margin | $\geq 45\%$ |

**GTN Margin Model**

$$M = 1 - \frac{R}{100} - \frac{C}{P}.$$

Constraint:

$$M \geq 0.45 \quad \Longleftrightarrow \quad \frac{R}{100} + \frac{C}{P} \leq 0.55.$$

Penalty:

$$\lambda \max(0, \frac{R}{100} + \frac{C}{P} - 0.55)^2.$$

**Leadership Translation:** This ensures the solver never chooses a price/rebate/copay combination that violates the required GTN margin floor. Bad configurations simply become "too expensive" for the solver to select.

# 6 Implementation Using Ocean SDK

## 6.1 Python Example

The following code demonstrates how to implement a simple QUBO model using the D-Wave Ocean SDK:

```python
from dwave.samplers import SimulatedAnnealingSampler
from dimod import BinaryQuadraticModel

# Define QUBO dictionary (example for gas station pricing)
# q0, q1, q2 represent binary encoding of price increments
Q = {
    ('q0', 'q0'): -1.2,  # Linear term for q0
    ('q1', 'q1'): -0.8,  # Linear term for q1
    ('q2', 'q2'): -0.4,  # Linear term for q2
    ('q0', 'q1'): 0.4,   # Quadratic interaction
    ('q1', 'q2'): 0.3,   # Quadratic interaction
}

# Create BinaryQuadraticModel from QUBO
bqm = BinaryQuadraticModel.from_qubo(Q)

# Initialize classical sampler
sampler = SimulatedAnnealingSampler()

# Sample solutions
samples = sampler.sample(bqm, num_reads=500)

# Extract best solution
best_solution = samples.first
print(f"Best solution: {best_solution.sample}")
print(f"Energy: {best_solution.energy}")

# Convert binary solution back to price
def decode_price(solution, base_price=2.50, increment=0.01):
    """Convert binary variables back to actual price"""
    binary_value = (solution.get('q0', 0) +
                    2 * solution.get('q1', 0) +
                    4 * solution.get('q2', 0))
    return base_price + increment * binary_value

optimal_price = decode_price(best_solution.sample)
print(f"Optimal price: ${optimal_price:.2f}")
```

Listing 1: Basic QUBO Implementation with Ocean SDK

## 6.2 Advanced Constraint Handling

For more complex financial models with multiple constraints, the following pattern is recommended:

```python
import numpy as np
from dimod import BinaryQuadraticModel

class FinancialQUBO:
    def __init__(self):
        self.Q = {}
        self.penalty_weights = {}

    def add_objective_term(self, var1, var2=None, weight=1.0):
        """Add objective terms to QUBO"""
        if var2 is None:
            var2 = var1
        self.Q[(var1, var2)] = self.Q.get((var1, var2), 0) + weight

    def add_constraint_penalty(self, constraint_type, variables,
                               target, penalty_weight=10.0):
        """Add penalty terms for constraint violations"""
        if constraint_type == 'equality':
            # (sum(vars) - target)^2 expansion
            for i, var1 in enumerate(variables):
                for j, var2 in enumerate(variables):
                    if i <= j:
                        coeff = penalty_weight * (2 if i == j else 1)
                        self.add_objective_term(var1, var2, coeff)

            # Linear terms: -2*target*sum(vars)
            for var in variables:
                self.add_objective_term(var, var, -2 * target * penalty_weight)

    def build_bqm(self):
        """Convert to BinaryQuadraticModel"""
        return BinaryQuadraticModel.from_qubo(self.Q)

# Example usage for portfolio optimization
def create_portfolio_qubo(num_assets=3, target_allocation=1.0):
    qubo = FinancialQUBO()

    # Asset allocation variables (binary encoded)
    variables = []
    for asset in range(num_assets):
        for bit in range(4):  # 4 bits per asset
            variables.append(f'a{asset}_b{bit}')

    # Budget constraint: sum of allocations = 1.0
    # This is simplified - actual implementation would need
    # proper binary-to-decimal conversion
    budget_vars = [f'a{i}_total' for i in range(num_assets)]
    qubo.add_constraint_penalty('equality', budget_vars,
                                target_allocation, penalty_weight=50.0)

    return qubo.build_bqm()

# Create and solve portfolio optimization
portfolio_bqm = create_portfolio_qubo()
sampler = SimulatedAnnealingSampler()
portfolio_solution = sampler.sample(portfolio_bqm, num_reads=1000)

print("Portfolio optimization complete")
```

```
print(f"Best energy: {portfolio_solution.first.energy}")
```

Listing 2: Advanced QUBO with Constraint Penalties

# 7 Deployment Considerations

## 7.1 DAYSTROM Node Configuration

For production deployment on hybrid quantum-classical systems:

- Use `dwave.cloud.client` for QPU access
- Implement fallback to classical solvers for reliability
- Monitor solution quality metrics and convergence
- Set appropriate annealing parameters based on problem size

## 7.2 Performance Optimization

- Pre-scale variables to avoid numerical precision issues
- Use problem decomposition for large-scale models
- Implement solution caching for repeated similar problems
- Validate constraint satisfaction in post-processing

# 8 Conclusion

This framework provides a systematic approach to encoding financial optimization problems as QUBO models. The six examples demonstrate broad applicability across industries, while the Python implementations show practical deployment paths.

Key benefits include:

- Unified handling of discrete financial decisions
- Natural constraint incorporation through penalties
- Compatibility with emerging quantum optimization hardware
- Clear translation between business requirements and mathematical models

Future work will extend these methods to multi-period planning and stochastic optimization scenarios.